Private Set Operations from Circuit-Based PSI

Jiseung Kim, Hyung Tae Lee, and Yongha Son

***Abstract*—Private set operations (PSOs) have been extensively studied due to their wide range of applications. In many practical scenarios, it is essential to perform advanced computations on the set operation result while maintaining the security. However, existing PSO protocols for advanced computations often suffer from the during-execution leakage, allowing an adversary to extract additional information beyond the final operation result.**

In this paper, we propose a novel framework for constructing PSO protocols by leveraging circuit-based private set intersection (circuit-PSI) in a black-box manner, enabling a range of advanced operations on the set intersection result while mitigating information leakage. Specifically, we present PSI variants, including PSI-Sum, which computes the sum of associated values; PSI-Card, which determines the cardinality of the intersection; PSI-Threshold, which evaluates whether the intersection cardinality exceeds a given threshold; and PSI-InnerProd, which computes the inner product of vectors associated with the intersection. We also provide a private set union (PSU) protocol based on circuit-PSI without the during-execution leakage.

To evaluate the efficiency of our proposed protocols, we provide implementation results of ours and existing PSO protocols closely related to our work. The experimental results demonstrate that our proposed protocols achieve performance comparable to state-of-the-art methods, while resolving the during-execution leakage that the state-of-the-art methods suffer from. Furthermore, as a unique feature of our proposals, our protocols can significantly reduce the online protocol execution time by separating time-consuming setup phase, which comes from the preprocessing optimization of circuit-PSI. In terms of online execution time, PSI-Card and PSI-Sum show a 9.92–13× improvement over prior work, while PSU achieves a 4.8–5.2× speedup. Additionally, PSI-InnerProd demonstrates a remarkable 47–56× speedup over the previous result over a LAN network environment.

***Index Terms*—Private set operations, circuit-PSI, private computation over set intersection, private set union**

I. Introduction

PRIVATE set operations (PSOs) enable parties, each possessing their own datasets, to collaboratively compute set operation results while ensuring that no additional information beyond the final outcome is disclosed. This capability has broad applications in various domains, including privacy-preserving contact tracing [1], privacy-preserving data analysis [2], and cyber risk assessments [3], where sensitive information must be processed without compromising individual data privacy. Due to their significance, PSOs have garnered considerable attention from the cryptography and security communities, leading to active research on various types of PSOs [4]–[8]. Among these, this work focuses specifically on PSO protocols within a two-party setting.

One of the most well-known examples is private set intersection (PSI), in which one of the two parties obtains the intersection set. PSI is arguably one of the most extensively studied PSOs, and its performance has improved at a remarkably rapid pace over the past decade [9]–[13]. However, this high performance is achievable because of the simplicity of the goal of PSI; these advances in PSI have not been directly applied to advanced set operations that aim to achieve more complicated results.

Circuit-based PSI (circuit-PSI), originally proposed by Huang et al. [14], may offer a potential solution for designing advanced PSO protocols. It allows both parties to obtain secret-shared or encrypted membership results at the end of the protocol. This design enables the parties to perform additional secure two-party computation (2PC) steps on the obtained results, significantly enhancing the flexibility. Moreover, since the parties possess secret shares of the membership bits, this approach can be naturally extended to support many other operations beyond basic set intersection. However, to the best of our knowledge, there has been no known attempt to design PSO protocols by leveraging a circuit-PSI protocol as a core building block in a fully black-box manner.

An alternative research direction leverages the *reverse private membership test* (RPMT) [6], [15], in which a server learns, for each element held by a client, whether it belongs to the server's set Y. At the end of the protocol, the server receives only a binary vector $(e_1, \ldots, e_n)$, where each e_i is set to 1 if $x_i \in Y$ and 0 otherwise for every x_i in the client's set X. Note that the term "reverse" indicates that the party holding Y (e.g., the server)–rather than the party holding X (e.g., the client)–learns the membership results. Despite this advantage, the server remains unaware of the exact intersection $X \cap Y$, since it lacks knowledge of the order of elements in X. This property makes RPMT particularly useful for PSOs [7], when combined with additional cryptographic primitives, such as oblivious transfer (OT). Moreover, this approach demonstrates excellent practical performance, particularly in classical Diffie-Hellman (DH)-based framework [8] for some set operations.

However, a critical drawback of RPMT-based PSOs is their inherent partial information leakage, which is undesirable in many scenarios. More precisely, these protocols typically follow a two-phase structure: In the *first step*, the protocol executes RPMT, revealing membership bits. In the *second step*, these membership bits are used for further computations. This multi-step design introduces a potential *during-execution leakage* [16], where an adversary can intentionally *abort* the

J. Kim is with Department of Computer Science and Artificial Intelligence, Jeonbuk National University, Jeonju 54896, Republic of Korea (email: jiseungkim@jbnu.ac.kr).

H. T. Lee is with School of Computer Science and Engineering, Chung-Ang University, Seoul 06974, Republic of Korea (email: hyungtaelee@cau.ac.kr).

Y. Son is with Department of Convergence Security Engineering, Sungshin Women's University, Seoul 02844, Republic of Korea (yongha.son@sungshin.ac.kr).

J. Kim and H. T. Lee equally contributed to this paper and Y. Son is the corresponding author. This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No. RS-2024-00399491, Development of Privacy-Preserving Multiparty Computation Techniques for Secure Multiparty Data Integration).

protocol after the first step, thereby gaining knowledge of the intersection's partial information without completing the protocol. For instance, in a threshold private set intersection (PSI-Threshold) protocol, the desired output is merely a Boolean value indicating whether $|X \cap Y| \geq t$ for a (pre-determined) threshold t, without revealing the exact intersection size. However, an RPMT-based approach inadvertently discloses $|X \cap Y|$ as an intermediate result. Similarly, in RPMT-based PSI-Sum and PSI-InnerProd protocols, the intersection cardinality is inherently revealed. Note that, in a PSI-Sum protocol, the parties collaboratively compute the sum of the associated values corresponding to the elements in the intersection. Meanwhile, in a PSI-InnerProd protocol, the parties jointly compute an inner product between two vectors, where each vector consists of the associated values corresponding to the elements in the intersection. In both cases, the leakage of $|X \cap Y|$ provides more information than strictly necessary. To mitigate this issue, Jia et al. [16] proposed a solution that maintains RPMT outputs in secret shares. However, their approach suffers from significant communication overhead compared to classical RPMT-based solutions, making it impractical in many real-world scenarios. Specifically, for privately computing the union of two sets, each of size 2^{20}, the method proposed in [16] requires about $2,430$ MB of communication, whereas the state-of-the-art RPMT-based approach [8] achieves the same task with only 103.3 MB of communication.

A. Our Results

Although RPMT-based methods offer simplicity and often strong performance for specific set operations, they inevitably reveal the partial information of the intersection result and remain vulnerable to during-execution leakage. This limitation motivates the need for an alternative foundational protocol for advanced PSOs–one that prevents the unintended disclosure of additional information beyond what is required by their ideal functionalities.

On the other hand, circuit-PSI was historically considered impractical for such tasks due to the perceived computational overhead of encoding additional operations within Boolean circuits. However, recent advancements have significantly improved its scalability, making it a viable approach for more complex PSO tasks while maintaining computational efficiency. A key advantage of circuit-PSI is that it inherently produces membership bits in a secret-shared form, unless they are explicitly reconstructed. This crucial property *prevents the leakage of extra information*, granting protocol designers finer control over what details are revealed in the computation.

In this paper, we demonstrate how circuit-PSI can serve as a versatile foundation for a broad range of PSOs, often achieving security and performance levels that match or even exceed those of RPMT-based approaches. By treating a circuit-PSI protocol as a "black-box" manner, followed by minimal post-processing steps implemented via standard OT or 2PC protocols, we systematically construct the following protocols:

- **PSI-Sum** and **PSI-Cardinality** which jointly computes the sum of associated values and the cardinality of the intersection, respectively, without revealing additional information beyond the result (Section III-A and Section III-B).
- **PSI-Threshold** which determines $|X \cap Y| \geq t$ without exposing additional overlap details (Section III-C).
- **PSI-InnerProd** which enables inner product computations confined to intersecting elements (Section IV).
- **Private Set Union (PSU)** which prevents the cardinality leakage commonly seen in other existing PSU approaches, where PSU collaboratively computes the union of two parties (Section V).

Additionally, we present implementation results of ours and existing PSO protocols which are closely related to our work in Section VI. The results show that our PSI-Card, PSI-Sum, and PSU protocols achieve performance comparable to those in [8] in terms of total running time, particularly when communication overhead has a relatively small impact on performance–for instance, when the dataset size is around 2^{16} in a LAN network environment. However, for large dataset sizes, such as 2^{20}, and in scenarios with slower network conditions, our protocols show a slight performance overhead compared to other existing protocols. Even so, we emphasize again that, unlike the protocols in [8], our protocols remain resistant to during-execution leakage, providing a crucial security advantage. Furthermore, for PSI-InnerProd and PSI-Threshold, our protocols achieve the best performance outperforming existing approaches.

Particularly, our proposed protocol can be preprocessed by shifting random OT generation parts into the setup phase in a circuit-PSI implementation and this process reduces the online running times of our protocol significantly. For example, compared to the previous best results in [8], PSI-Card and PSI-Sum a 9.92–13$\times$ improvement in online execution time, while PSU demonstrates a 4.8–5.2$\times$ speedup. Moreover, PSI-InnerProd shows a remarkable 47–56$\times$ improvement over the results in [17] over a LAN network environment.

B. Concurrent and Independent Work

Recently, Chandran et al. [18] proposed a modular construction for PSU that also leverages circuit-PSI. In the sense that it treats circuit-PSI as a black box, their work is arguably the closest to ours among existing works. However, at a high level, their protocol still follows an RPMT-based two-step structure–(1) executing an RPMT-like functionality and then (2) running a post-processing procedure. As a result, their approach remains susceptible to the during-execution leakage identified by Jia et al. [16].

C. Related Works

Since the first PSO protocol for intersection operation was proposed by Meadows [19], numerous approaches have been proposed to enhance the design and efficiency of PSO protocols. Among them, this subsection focuses on circuit-PSI protocols, private computation over the intersection result, and private set union protocols, which are closely relevant to the contributions of our work.

Circuit-PSI. Circuit-PSI can be traced back to the work of Huang et al. [14], who proposed to encode the equality check for each pair (x, y) in a Boolean circuit. Their construction leveraged garbled circuits and achieved $O(n \log n)$ complexity where n is the size of the input set. Subsequently, Pinkas et al. [20] improved the communicational complexity to $O(n)$, but requires $O(n \log n^2)$ computational complexity. Later, Chandran et al. [21] proposed an efficient circuit-PSI protocol with linear computational and communicational complexity. We note that the protocols of Pinkas et al. and Chandran et al. are both designed using OT extension based on oblivious pseudorandom functions (OPRFs). In contrast, the use of vector oblivious linear evaluation (VOLE) could improve the computational efficiency of circuit-PSI protocols. Rindal and Schoppmann [11] introduced a circuit-PSI protocol based on VOLE, which was later improved by Raghuraman and Rindal [12] through the use of subfield VOLE. More recently, Bienstock et al. [13] proposed a circuit-PSI protocol that leverages oblivious key value store (OKVS) to further improve performance. An important appeal of circuit-PSI is that it can emit secret-shared or encrypted membership results, making it straightforward to append further secure two-party computation (2PC) steps. Thus, while prior literature focused on circuit-PSI primarily for computing intersections only, its inherent flexibility suggests it could be extended to many other operations once parties hold the secret shares of membership bits.

Private Computation over the Intersection Result. In this paper, we propose PSI-Sum, PSI-Cardinality, PSI-Threshold, and PSI-InnerProd protocols. For PSI-Sum, Ion et al. [22] introduced a generic transformation that converts existing PSI protocols under specific settings into PSI-Sum protocols. However, their approach leverages additive homomorphic encryption (AHE) schemes, resulting in significant computational and communication overhead. Later, Garimella et al. [7] proposed a PSI-Sum protocol based on a variant of RPMT. While their construction improves the efficiency, it operates under a slightly different definition of PSI-Sum. Very recently, Chen et al. [8] presented an improved PSI-Sum protocol based on RPMT under the classical DH-based framework. However, their construction suffers from during-execution leakage.

For PSI-InnerProd, Lepoint et al. [23] introduced the inner product private join and compute functionality, which is conceptually equivalent to PSI-InnerProd, and provided an instantiation of the protocol. Their construction is efficient in settings where the receiver's dataset is significantly smaller than the sender's. However, its efficiency degrades when the input sizes are balanced. To address this, Chida et al. [17] proposed an improved PSI-InnerProd protocol by introducing a new variant of OPRF, which achieves better performance across a wide range of parameter settings, but it also suffers from during-execution leakage.

Finally, for PSI-Threshold, to the best of our knowledge, there has been no prior work that realizes this functionality.

Private Set Union. The first PSU protocol was introduced by Kissner and Song [4], who designed a protocol by representing sets as polynomials and encrypting them using AHE. Following this work, several PSU protocols [5], [24]–[26] based on AHE schemes were developed to improve efficiency. However, this line of researches suffers from inherent communicational inefficiencies, due to the overhead introduced by polynomial representations and the limitations of AHE schemes. Recently, Kolesnikov et al. [6] and subsequently Jia et al. [3] proposed efficient PSU protocols from symmetric-key techniques. Later, Zhang et al. [15] and Chen et al. [8] presented PSU protocols using the RPMT technique that achieve $O(n)$ communication complexity. However, as discussed earlier in the introduction, these constructions are vulnerable to during-execution leakage. To address this issue, Jia et al. [16] proposed a solution by incorporating symmetric-key operations, but it incurs significantly higher communication costs, limiting its practicality. As a concurrent and independent work to ours, Chandran et al. [18] proposed a new PSU protocol based on circuit-PSI, but it follows an RPMT-like two-step structure and so it is susceptible to the during-execution leakage.

D. Organization of the Paper

The following section introduces preliminaries and building blocks that will be utilized in our protocols. In Section III, we provide our proposed PSI variants based on circuit-PSI, including PSI-Sum, PSI-Card, and PSI-Threshold. Next, we present our PSI-InnerProd protocol in Section IV and our PSU protocol in Section V. The implementation results of our proposed PSO protocols, along with comparisons to existing related works are given in Section VI.

II. Preliminaries and Building Blocks

In this section, we present definitions of private set operations and fundamental building blocks used in our constructions. We begin by introducing several notations that will be used throughout the paper.

A. Notations

We represent vectors as bold lowercase letters. For any real number x, we denote $\lfloor x \rceil$ as the rounded value of x to the nearest integer. The i-th component of a vector $\mathbf{v}$ is denoted by v_i or $v[i]$. For an integer k, we denote the set $\{1, \cdots, k\}$ by $[k]$. The logarithm function $\log$ is assumed to have base 2 without special mention. For any Boolean statement T, i.e., a statement that is either true or false, we define $\mathbf{1}(T)$ as its truth value, where $\mathbf{1}(T) = 1$ if T is true and $\mathbf{1}(T) = 0$ otherwise. For a set X, $|X|$ indicates the cardinality of X. For a prime power integer $q \geq 2$, we denote by $\mathbb{F}_q$ a finite field of size q. We consider the following two types of secret-shares.

- Boolean shares of a bit-string $b \in \{0,1\}^*$: Two bit-strings b_0 and b_1 satisfying $b_0 \oplus b_1 = b$, where $\oplus$ is the XOR operation.
- Arithmetic shares of an element $v \in \mathbb{F}$: Two elements v_0 and v_1 satisfying $v_0 + v_1 = v$, where $+$ is the addition operation in the field $\mathbb{F}$.

B. Security Notion

We use a standard notion of security against semi-honest adversaries and provide a simplified description for the 2-party case. Consider a two-party protocol Π that computes an ideal functionality $f(x_1, x_2)$ where party P_i has input x_i. For each party P_i, let $\mathsf{view}_i(x_1, x_2)$ denote the view of P_i during an execution of Π on input x_1, x_2. Specifically, it consists of the input x_i, the messages sent or received, the random tape sampled during the protocol execution, and the output of the protocol Π.

Definition 1. *We say that a (two-party) protocol Π for P_1 and P_2 computes f securely against semi-honest adversaries if there exist simulators Sim_1 and Sim_2 such that for any inputs x_1, x_2 of P_1 and P_2, respectively,*

$$\mathsf{Sim}_i(x_i, f(x_1, x_2)) \cong_c \mathsf{view}_i(x_1, x_2)$$

for $i = 1, 2$ where $\cong_c$ represents computational indistinguishability.

C. Oblivious Transfers

This subsection provides the formal definition of the oblivious transfer (OT) protocol and introduces several fundamental protocols built on OT.

In OT, the sender takes two messages m_0 and m_1 as inputs, and lets the receiver obtain a message m_b for a choice bit $b \in \{0, 1\}$, while the sender cannot learn any information about the receiver's choice b, and the receiver cannot learn any information about the other message m_{1-b}. The ideal functionality of OT is presented in Fig. 1.

Parameters: A message bit-length ℓ **Input:** $\mathcal{S}$ with messages $m_0, m_1 \in \{0,1\}^\ell$ and $\mathcal{R}$ with a choice bit $b \in \{0, 1\}$ **Functionality:** Output m_b to $\mathcal{R}$.

Fig. 1: Ideal functionality $\mathcal{F}_{\mathsf{OT}}$ of OT

Several variants of the OT protocol have been proposed. Among them, we introduce protocols that will be used throughout this paper. The random OT protocol differs from the standard OT in that the sender provides no input and receives two random OT messages r_0, r_1,. The receiver's input and output remain the same as in the standard OT.

Boolean to Arithmetic Conversion. OT can be used to convert Boolean shares into arithmetic shares, which we refer to as B2A-OT hereafter. Suppose there are two parties, a sender $\mathcal{S}$ and a receiver $\mathcal{R}$. $\mathcal{S}$ holds a bit b_s and a value v in a field $\mathbb{F}$, while $\mathcal{R}$ holds a bit b_r such that $b = b_s \oplus b_r$. The two parties then engage in OT to obtain arithmetic shares of $b \cdot v$ as follows: $\mathcal{S}$ samples a random element $a_s \xleftarrow{\$} \mathbb{F}$ and sets the OT messages as $m_{b_s} = -a_s$ and $m_{1-b_s} = v - a_s$. $\mathcal{R}$ sets the OT choice as b_r. After the execution of OT, $\mathcal{R}$ receives the OT message m_{b_r} and sets it as a_r. It can be easily verified that

$$a_s + a_r = (b_s \oplus b_r) \cdot v = b \cdot v,$$

which means that a_s and a_r are arithmetic shares of $b \cdot v$.

Generating Multiplication Shares. OT can also be used to generate arithmetic shares of the multiplication of two elements v, w in a field $\mathbb{F}$. The idea is to consider a bit-representation of w, say $w_{k-1} \cdots w_0$, and utilize the equality

$$v \cdot w = v \sum_{i=0}^{k-1} 2^i w_i = \sum_{i=0}^{k-1} 2^i v \cdot w_i$$

where $k = \log |\mathbb{F}|$.

Suppose that two parties hold $v \in \mathbb{F}$ and $w \in \mathbb{F}$, respectively. They establish arithmetic shares of each summand $2^i v \cdot w_i$ by engaging in OT, where the sender's OT messages are $m_0 = -r_i, m_1 = 2^i v - r_i$ for a randomly sampled $r_i \xleftarrow{\$} \mathbb{F}$ and the receiver's OT choice is w_i. By summing all shares of the summands, the two parties obtain arithmetic shares of $v \cdot w$.

GMW Protocol: Boolean Circuit Evaluation. Any Boolean circuit can also be securely evaluated using OT, which is also known as GMW protocol [27]. Since any Boolean circuit can be represented with XOR and AND gates, it only suffices to specify how to evaluate XOR and AND gate. For that, suppose that two parties hold Boolean shares (x_s, y_s) and (x_r, y_r) of bits x and y, respectively. The Boolean shares of the XOR operation $x \oplus y$ can be computed locally by evaluating $x_s \oplus y_s$ and $x_r \oplus y_r$ independently. The Boolean shares of the AND operation $x \wedge y$ can be computed by two instances of OT, based on the following relation:

$$\begin{aligned} x \wedge y &= (x_s \oplus x_r) \wedge (y_s \oplus y_r) \\ &= (x_s \wedge y_s) \oplus (x_s \wedge y_r) \oplus (x_r \wedge y_s) \oplus (x_r \wedge y_r). \end{aligned}$$

In the above relation, $x_s \wedge y_s$ and $x_r \wedge y_r$ can be evaluated locally by the sender and the receiver, respectively. For $x_s \wedge y_r$, the two parties engage in OT, where the sender's OT messages are $m_0 = r_1, m_1 = r_1 \oplus x_s$ for a random bit r_1 chosen by the sender and the receiver's OT choice is y_r. After the OT execution, the sender obtains r_1, while the receiver obtains $r_1 \oplus (x_s \wedge y_r)$. Similarly, for $x_r \wedge y_s$, they execute another instance of OT, with the sender's OT messages $m_0 = r_2, m_1 = r_2 \oplus x_r$ and the receiver's OT choice y_s, where r_2 is a random bit selected by the sender. After the OT execution, the sender obtains r_2, while the receiver obtains $r_2 \oplus (x_r \wedge y_s)$. Finally, the sender and the receiver compute their shares x' and y' of $x \wedge y$ as

$$\begin{aligned} x' &= r_1 \oplus r_2 \oplus (x_s \wedge y_s), \\ y' &= m_{y_r} \oplus m_{x_r} \oplus (x_r \wedge y_r) \\ &= (r_1 \oplus (x_r \wedge y_s)) \oplus (r_2 \oplus (x_s \wedge y_r)) \oplus (x_r \wedge y_r). \end{aligned}$$

D. Oblivious Pseudorandom Function (OPRF)

A pseudorandom function (PRF) F is a deterministic function that takes a key k and an element x as inputs, with outputs that are indistinguishable from those of a truly random function. An oblivious PRF (OPRF) protocol is a two-party protocol between a sender $\mathcal{S}$ and a receiver $\mathcal{R}$. $\mathcal{S}$ holds the PRF key k,while $\mathcal{R}$ holds the element x. After executing the OPRF protocol, $\mathcal{R}$ obtains the evaluation result $F_k(x)$, while $\mathcal{S}$ learns nothing about x.

We denote $\mathcal{F}_{\mathsf{PRF}}$ as the ideal functionality of a PRF, which takes the PRF key k and an element x as inputs and returns the evaluation result $F_k(x)$. The ideal functionality of the OPRF protocol is presented in Fig. 2.

Parameters: A PRF function F and the bit-length ℓ of its output
Input: $\mathcal{S}$ with a PRF key k and $\mathcal{R}$ with an input x
Functionality: Output $F_k(x)$ to $\mathcal{R}$.

Fig. 2: Ideal functionality $\mathcal{F}_{\mathsf{OPRF}}$ of OPRF

E. Circuit-PSI

Circuit-based private set intersection (Circuit-PSI) is a functionality that outputs a membership information in a secret-shared manner [11], [12], [28]. The ideal functionality of CPSI is given in Fig. 3.

Parameters: A set size n, an output size β, and a randomized function Reorder that maps a set X of size n to an injective function $\mathsf{idx} : X \to [\beta]$.
Input: $\mathcal{S}$ with a set X and $\mathcal{R}$ with a set Y.
Functionality: Compute an injective function $\mathsf{idx} \leftarrow \mathsf{Reorder}(X)$ and a membership vector $\mathbf{b} \in \{0,1\}^\beta$ defined as

$$b_i = \begin{cases} \mathbf{1}(x \in Y) & \text{if } \mathsf{idx}(x) = i, \\ 0 & \text{otherwise.} \end{cases}$$

Send idx to $\mathcal{R}$, and Boolean shares of $\mathbf{b}$, say $\mathbf{b}_s$ and $\mathbf{b}_r$ such that $\mathbf{b} = \mathbf{b}_r \oplus \mathbf{b}_s$ to each party.

Fig. 3: Ideal functionality $\mathcal{F}_{\mathsf{cPSI}}$ of circuit-PSI

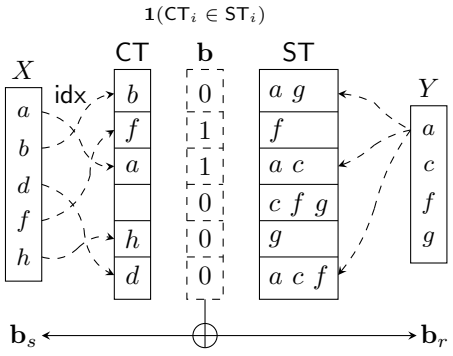

Fig. 4: An example of circuit-PSI: One party with input set X performs cuckoo hashing that determines the output ordering map idx, while the other party with input set Y performs simple hashing. As a result of circuit-PSI, two parties obtain the Boolean shares of the membership vector $\mathbf{b} = \mathbf{b}_s \oplus \mathbf{b}_s$.

In Fig. 3, the reordering function Reorder stems from the use of cuckoo hashing [29], which is essential to achieve linear complexity [11]. More precisely, the construction of CPSI begins by having two parties build hash tables for their respective input sets using the following special hashing method: One party with input set X employs multiple hash functions–typically three–to ensure that each bin contains at most one item. This process is called cuckoo hashing. The other party with the input set Y applies all hash functions to load the items into the bins, a process referred to as simple hashing to distinguish it from cuckoo hashing. As a result, the original set membership test $x \in Y$ reduces to the bin membership test $\mathsf{CT}_i \in \mathsf{ST}_i$, which is one of key techniques in state-of-the-art CPSI constructions to improve efficiency. A visual example in Fig. 4 may help in understanding.

III. Extension to Variants of Private Set Intersection

In this section, we present variants of the PSI protocol–PSI-Sum, PSI-Card, and PSI-Threshold—whose key idea is to perform B2A-OT on the output of circuit-PSI. More precisely, these operations can be commonly understood as follows: The sender, who holds X, also holds associated data $D = \{d_x \in \mathbb{F} : x \in X\}$. Using B2A-OT, the Boolean shares of $\mathbf{b} \in \{0,1\}^\beta$ are converted into the arithmetic shares of the data D. It is worth noting that, since the sender knows the index map idx, it can reorder the data with respect to the order of $\mathbf{b}$. Thus, for each $i \in [\beta]$, the sender can determine the corresponding data $d_x \in D$, which enables two parties to jointly perform B2A-OT to generate arithmetic shares of $\mathbf{a} = (b_i \cdot d_x)$. Fig. 5 shows a visual explanation of this procedure.

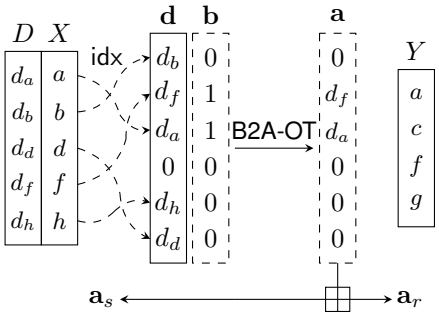

Fig. 5: Arithmetic shares of the associated data of the X-holder

A. PSI-Sum

As the most direct application of Fig. 5, we consider the PSI-Sum protocol. The ideal functionality of PSI-Sum is described in Fig. 6.

Parameters: A set size n and a data field $\mathbb{F}$
Input: $\mathcal{S}$ with a set X and an associated data $D = \{d_x \in \mathbb{F} : x \in X\}$, and $\mathcal{R}$ with a set Y
Functionality: $\mathcal{R}$ obtains $S_{X \cap Y} := \sum_{x \in X \cap Y} d_x$.

Fig. 6: Ideal functionality of PSI-Sum

After two parties obtain $\mathbf{a}_s$ and $\mathbf{a}_r$ by running the circuit-PSI protocol, they simply sum up all components of the vectors to obtain arithmetic shares of $S_{X \cap Y} = \sum_{x \in X \cap Y} d_x$. More

precisely, the sender and the receiver can locally compute $a_s := \sum_{i=1}^{\beta} a_{s,i}$ and $a_r := \sum_{i=1}^{\beta} a_{r,i}$, respectively, which satisfy $a_s + a_r = S_{X\cap Y}$ over $\mathbb{F}$. Finally, the desired sum $S_{X\cap Y}$ can be opened to either party by having the other party send their share. The detailed description is given in Fig. 7.

Parameters: A set size n and a data field $\mathbb{F}$.
Input: $\mathcal{S}$ with a set X and an associated data $D = \{d_x \in \mathbb{F} : x \in X\}$, and $\mathcal{R}$ with a set Y.
Protocol:

1) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\mathsf{cPSI}}$ and obtain secret shares $\mathbf{b}_s$ and $\mathbf{b}_r$, respectively, such that $\mathbf{b} = \mathbf{b}_r \oplus \mathbf{b}_s \in \{0,1\}^\beta$. $\mathcal{S}$ additionally obtains a mapping $\mathsf{idx} : X \to [\beta]$.
2) $\mathcal{S}$ defines $\mathbf{d} \in \mathbb{F}^\beta$ such that for each $i \in [\beta]$
$$d_i = \begin{cases} d_x & \text{if } \mathsf{idx}(x) = i \\ 0 & \text{otherwise.} \end{cases}$$
3) For each $i \in [\beta]$, $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathsf{B2A\text{-}OT}$:
   - $\mathcal{S}$ inputs $b_{s,i} \in \{0,1\}$, $d_i \in \mathbb{F}$ and obtains $a_{s,i} \in \mathbb{F}$.
   - $\mathcal{R}$ inputs $b_{r,i} \in \{0,1\}$ and obtains $a_{r,i} \in \mathbb{F}$.
4) $\mathcal{S}$ computes $a_s = \sum_i a_{s,i}$ and $\mathcal{R}$ computes $a_r = \sum_i a_{r,i}$.
5) $\mathcal{S}$ sends a_s to $\mathcal{R}$.
6) $\mathcal{R}$ outputs $a_s + a_r$.

Fig. 7: Our PSI-Sum protocol based on circuit-PSI

We remark that the protocol can be easily modified so that $\mathcal{S}$ obtains $S_{X\cap Y}$ by having $\mathcal{R}$ send a_r to $\mathcal{S}$. Furthermore, if $\mathcal{R}$ is the party obtaining the result, a small optimization is possible: $\mathcal{S}$ samples $a_{s,i}$ so that $a_s = \sum_i a_{s,i} = 0$. In this case, $a_r = \sum_i a_{r,i}$ directly becomes $\sum_{x\in X\cap Y} d_x$, which removes the need to send a_s and so reduces one interaction.

B. PSI with Cardinality (PSI-Card)

The purpose of PSI-Card is to compute the cardinality of the intersection $X \cap Y$. The ideal functionality of PSI-Card is presented in Fig. 8. It can be directly obtained as a special case of PSI-Sum, where the associated data is set to $d_x = 1$ for every $x \in X$.

Parameters: A set size n
Input: $\mathcal{S}$ with a set X and $\mathcal{R}$ with a set Y
Functionality: $\mathcal{R}$ obtains $|X \cap Y|$.

Fig. 8: Ideal functionality of PSI-Card

However, in order to complete the protocol description, it is necessary to specify the field $\mathbb{F}$ in which the associated data lies. Although the data itself is 1 for every $x \in X$, the sum of this data represents $|X \cap Y|$ as an integer. Thus, the field $\mathbb{F}$ should be sufficiently large to represent the arithmetic share of $|X \cap Y|$; equivalently, it suffices to set $|\mathbb{F}| \geq n$. The detailed protocol of PSI-Card is presented in Fig. 9.

Optimization: Reducing Communication Costs. We can optimize the PSI-Card protocol to reduce communication costs.

Parameters: A set size n
Input: $\mathcal{S}$ with a set X and $\mathcal{R}$ with a set Y
Protocol:

1) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\mathsf{cPSI}}$ and obtain secret shares $\mathbf{b}_s$ and $\mathbf{b}_r$, respectively, such that $\mathbf{b} = \mathbf{b}_r \oplus \mathbf{b}_s \in \{0,1\}^\beta$. $\mathcal{S}$ additionally obtains a mapping $\mathsf{idx} : X \to [\beta]$.
2) $\mathcal{S}$ and $\mathcal{R}$ agree on a field $\mathbb{F}$ with size larger than n.
3) $\mathcal{S}$ defines $\mathbf{d} \in \mathbb{F}^\beta$ such that for each $i \in [\beta]$
$$d_i = \begin{cases} 1 & \text{if } \mathsf{idx}(x) = i \\ 0 & \text{otherwise.} \end{cases}$$
4) For each $i \in [\beta]$, $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathsf{B2A\text{-}OT}$:
   - $\mathcal{S}$ inputs $b_{s,i} \in \{0,1\}$, $d_i \in \mathbb{F}$ and gets $a_{s,i} \in \mathbb{F}$.
   - $\mathcal{R}$ inputs $b_{r,i} \in \{0,1\}$ and gets $a_{r,i} \in \mathbb{F}$.
5) $\mathcal{S}$ computes $a_s = \sum_i a_{s,i}$ and $\mathcal{R}$ computes $a_r = \sum_i a_{r,i}$.
6) $\mathcal{S}$ sends a_s to $\mathcal{R}$.
7) $\mathcal{R}$ outputs $a_s + a_r$.

Fig. 9: Our PSI-Card protocol based on circuit-PSI

Observe the following relation for each i:

$$b_{s,i} \oplus b_{r,i} = (b_{r,i} - b_{s,i})^2 = b_{r,i} + b_{s,i} - 2b_{r,i}b_{s,i}.$$

From the above relation, we have

$$|X \cap Y| = \sum_i (b_{s,i} \oplus b_{r,i}) = \sum_i (b_{r,i} + b_{s,i} - 2b_{r,i}b_{s,i}).$$

Thus, instead of having $\mathcal{S}$ and $\mathcal{R}$ compute $\sum a_{s,i}$ and $a \sum_{r,i}$, respectively, obtained by invoking $\mathsf{B2A\text{-}OT}$, $\mathcal{S}$ and $\mathcal{R}$ can generate arithmetic shares a_s and a_r of $|X \cap Y|$ by calculating

$$a_s = \sum_i b_{s,i} + 2r_{i,0}, \tag{1}$$

$$a_r = -2\left(\sum_i r_{i,0} + b_{s,i}b_{r,i}\right) + \sum_i b_{r,i}, \tag{2}$$

where $r_{i,0}$ are random values generated during this process.

We realize the above using the random OT protocol. First, by invoking the random OT protocol β times, $\mathcal{S}$ obtains β pairs of elements $\{r_{i,0}, r_{i,1}\}_{i\in[\beta]}$ and $\mathcal{R}$ can access $\{r_{i,b_{r,i}}\}_{i\in[\beta]}$ without any leakage of $\{r_{i,1-b_{r,i}}\}_{i\in[\beta]}$. Next, for each $i \in [\beta]$, $\mathcal{S}$ sends an ℓ_0-bit message $T_i = r_{i,0} + r_{i,1} + b_{s,i}$ to $\mathcal{R}$. Then, $\mathcal{R}$ obtains one of the following based on $b_{r,i}$:

$$\begin{cases} r_{i,b_{r,i}} & \text{if } b_{r,i} = 0, \\ r_{i,0} + b_{s,i} & \text{if } b_{r,i} = 1. \end{cases}$$

We note that when $b_{r,i} = 0$ the first element can be directly retrieved from the previously conducted random OT protocol. When $b_{r,i} = 1$, the second element can be computed as $T_i - r_{i,b_{r,i}}$. By adjusting the representation of the obtained values, the result for $\mathcal{R}$ can be interpreted as $r_{i,0} + b_{s,i}b_{r,i}$ for each i.

As a result, $\mathcal{S}$ can obtain the set $\{b_{s,i} + 2r_{i,0}\}_{i\in[\beta]}$, while $\mathcal{R}$ can obtain the set $\{r_{i,0} + b_{s,i}b_{r,i}\}_{i\in[\beta]}$. By defining a_s and a_r as in Eq. (1) and (2), it obviously holds

$$|X \cap Y| = a_s + a_r.$$

Finally, $\mathcal{S}$ sends a_s to $\mathcal{R}$, which allows $\mathcal{R}$ to obtain $|X \cap Y|$. In the above modification, the communication from $\mathcal{S}$ to $\mathcal{R}$ consists of T_i and a_s, with a total communication cost of $\beta(\ell_0 + 1)$.

C. PSI-Threshold

We then explain how to construct a PSI-Threshold protocol, which allows $\mathcal{R}$ to obtain $\mathbf{1}(|X \cap Y| \geq t)$ without any other information, particularly $|X \cap Y|$. The ideal functionality of PSI-Threshold is given in Fig. 10.

Parameters: A set size n and a threshold t
Input: $\mathcal{S}$ with a set X and $\mathcal{R}$ with a set Y
Functionality: $\mathcal{R}$ obtains $\mathbf{1}(|X \cap Y| > t)$.

Fig. 10: Ideal functionality of PSI-Threshold

We would like to remark that the threshold t is considered as a secure input of $\mathcal{R}$ in our definition. Previously, PSI-threshold protocols were implicitly assumed to have t as a public parameter [30], since their proposals require that both parties know t. In contrast, our construction allows only one of two parties to know t, which could be useful for some real-world applications.

To design a PSI-Threshold protocol, we observe that in the middle of our PSI-Card protocol, $\mathcal{S}$ and $\mathcal{R}$ have arithmetic shares a_s and a_r of the cardinality of the intersection $|X \cap Y|$, while neither party knows the exact value of $|X \cap Y|$. At this point, we apply a generic two-party computation (2PC) protocol–particularly the GMW protocol [27] for our implementation–to securely compute $\mathbf{1}(|X \cap Y| > t)$. This completes the construction of our PSI-Threshold protocol. The details of our PSI-Threshold protocol are provided in Fig. 11.

Parameters: A set size n
Input: $\mathcal{S}$ with a set X, and $\mathcal{R}$ with a set Y and a threshold t
Protocol:

1) $\mathcal{S}$ and $\mathcal{R}$ invoke the PSI-Card protocol in Fig. 9, except for the final two steps, to obtain a_s and a_r.
2) $\mathcal{S}$ and $\mathcal{R}$ engage the GMW protocol on the Boolean circuit representation of $C(a_s, a_r, t) = \mathbf{1}((a_s + a_r) > t)$, with $\mathcal{S}$'s input $x = a_s$ and $\mathcal{R}$'s input $y = a_r$. As a result, $\mathcal{R}$ obtains the binary output.

Fig. 11: Our PSI-Threshold protocol based on circuit-PSI

As a short remark on the cost of the GMW protocol, we note that it requires evaluating the Boolean representation of the arithmetic circuit $a_s + a_r > t$, which involves only one integer addition and a comparison of $\log n$-bit integers. This process requires at most $2 \log n$ AND gate evaluations, which is equivalent to $4 \log n$ instances of OT. The cost of this step is significantly smaller than other parts of the protocol.

D. Correctness and Security

For the correctness of the protocols, observe that $a_{s,i} + a_{r,i} = b_i \cdot d_i$ for every $i \in [\beta]$, and hence $a_s + a_r = \sum_{i \in [\beta]} b_i \cdot d_i$. From the definition of $\mathbf{b}$ and $\mathbf{d}$, the summation $\sum_{i \in [\beta]} b_i \cdot d_i$ becomes $S_{X \cap Y}$ in PSI-Sum (Fig. 7), and $|X \cap Y|$ in PSI-Card (Fig. 9).

The core of security is the fact that each party only obtains secret-shared information during the protocol execution, except the final desired output; for example, circuit-PSI computes the Boolean shares of membership vector $\mathbf{b}$ and OT computes the arithmetic shares of associated data vector $\mathbf{d}$. Thus, assuming the security of circuit-PSI and OT that used to generate the shares $\mathbf{b}$ and $\mathbf{d}$, our proposed protocols also become secure.

IV. Extension to PSI-Inner Product

This section introduces the modular construction of the PSI-InnerProd protocol, whose ideal functionality is in Fig. 12. Similar to the previous section, the starting point is the circuit-PSI protocol, with a focus on a post-processing step using its outputs. However, unlike the previous protocols, which were relatively straightforward consequences of B2A-OT, our PSI-InnerProd protocol involves more complicated steps, as elaborated below.

Parameters: A set size n and a data field $\mathbb{F}$
Input: $\mathcal{S}$ with a set X and an associated data $D^X = \{d_x^X \in \mathbb{F} : x \in X\}$, and $\mathcal{R}$ with a set Y and an associated data $D^Y = \{d_y^Y \in \mathbb{F} : y \in Y\}$
Functionality: $\mathcal{R}$ obtains $\sum_{z \in X \cap Y} d_z^X \cdot d_z^Y$.

Fig. 12: Ideal functionality of PSI-InnerProd

A. Our PSI-InnerProd Protocol

We present our PSI-InnerProd protocol by highlighting the obstacles encountered when following a structure similar to other protocols in Section III and detailing key steps taken to overcome them.

1) Generate (Semi-)Shares of Associated Data of the Y-holder: The key difference between the PSI-InnerProd protocol and other PSO protocols in Section III is that $\mathcal{R}$ now also has associated data, denoted D^Y. Recall that each associated value of the X-holder is linked to a membership Boolean $(x \in Y)$ using the index map idx, which enables the X-holder to set the correct associated values when performing each B2A-OT that converts Boolean shares $(b_{s,i}, b_{r,i})$ into arithmetic shares $(a_{s,i}, a_{r,i})$. However, the Y-holder does not know which bin corresponds to each associated data in D^Y, so the same strategy cannot be applied.

To resolve this issue, we revisit the technique [28, Section 6] that generates a secret shares of data in D^Y in the middle of the circuit-PSI protocol.[1] That is, in addition to the membership

[1]The original description in [28] aims to generate Boolean shares of data in D^Y, but it can be converted to arithmetic shares without additional overhead.

shares $\mathbf{b}_s$ and $\mathbf{b}_r$, the two parties also respectively obtain vectors $\mathbf{a}_s^Y$ and $\mathbf{a}_r^Y$ such that

$$a_{r,i}^Y + a_{s,i}^Y = d_y^Y \text{ if } \mathsf{idx}(x) = i \text{ and } x = y \in Y.$$

A pictorial explanation of this procedure is presented in Fig. 13.

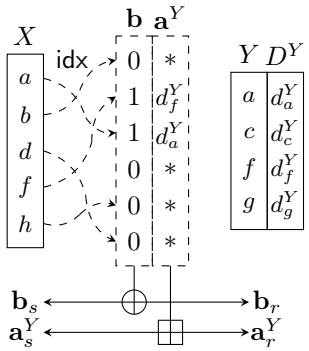

Fig. 13: (Semi-)Shares of Associted Data of the Y-holder

2) Clear Out to Zero Shares: It seems that the above step is sufficient to have arithmetic shares of associated data in D^Y. However, these shares are actually incomplete because they do not ensure $a_{r,i}^Y + a_{s,i}^Y = 0$ when $b_i = 0$, unlike the B2A-OT-based approach. To rectify this, we introduce an additional procedure using two calls of B2A-OT. Recall that B2A-OT takes an input Boolean shares $b \in \{0,1\}$ and some value $d \in \mathbb{F}$ as inputs, and generates shares of $b \cdot d$. Based on this fact, we use one B2A-OT call to generate arithmetic shares of $b_i \cdot a_{s,i}$ and the other to generate arithmetic shares of $b_i \cdot a_{r,i}$. By summing them, we obtain arithmetic shares of $b_i \cdot a$, which produces the complete vector shares:

$$a_{r,i}^Y + a_{s,i}^Y = \begin{cases} d_y^Y & \text{if } \mathsf{idx}(x) = i \text{ and } x = y \in Y, \\ 0 & \text{otherwise.} \end{cases}$$

A pictorial explanation of this procedure for clearing out to zero shares is provided in Fig. 14.

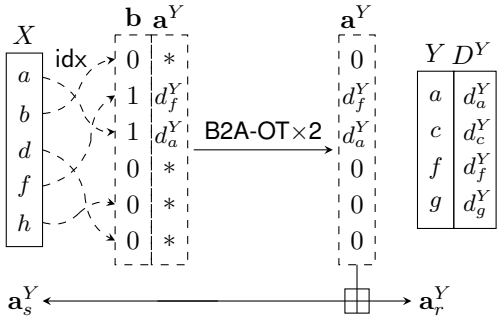

Fig. 14: Clear out to zero shares

We note that similar techniques have been presented in different contexts [23], [31], which are adapted to our case.

3) Multiply Shares with Associated Data of the X-holder: Finally, we proceed to convert the arithmetic shares of data in D^Y into arithmetic shares of data in $D^X \cdot D^Y$. Note that the X-holder can identify the correct position for each associated value in D^X. Thus, for each $i \in [\beta]$, we may assume that $\mathcal{S}$ knows the corresponding $d_i^X \in D^X$ (if any) as well as the arithmetic share of $d_i^Y \in D^Y$.

To specify the process, fix $i \in [\beta]$, and denote the corresponding X-side associated data as d^X and the Y-side associated data as d^Y that are secret-shared by $d^Y = d_s^Y + d_r^Y$. The X-holder can locally compute $d^X \cdot d_s^Y$, and hence it only remains to generate shares of $d^X \cdot d_r^Y$. This can be done by using MultShare, which completes the generation of arithmetic shares of $d^X \cdot d^Y$. A pictorial explanation of the above step for multiplying shares with the associated data of the X-holder is presented in Fig. 15.

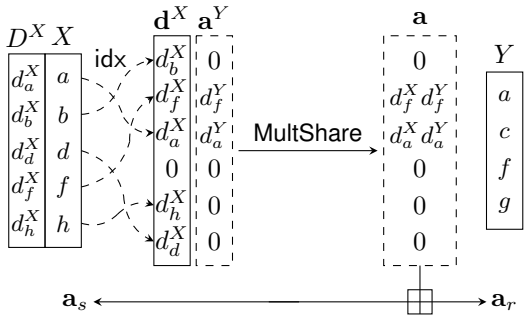

Fig. 15: Multiply shares with associated data of the X-holder

Finally, the two parties locally sum all components of the vector to obtain arithmetic shares of the inner product over $X \cap Y$. The full protocol description of our PSI-InnerProd can be found in Fig. 16.

B. Correctness and Security

The correctness of the protocols directly comes from the fact that

$$a_{s,i} + a_{r,i} = \begin{cases} b_i \cdot d_i^X \cdot d_y^Y & \text{if } \mathsf{idx}(x) = i \text{ and } x = y \in Y \\ 0 & \text{otherwise.} \end{cases}$$

Similarly to the previous section protocols, the core of security is the fact that each party only obtains secret-shared information during the protocol execution, while every secret shares computation is done by secure functionality such as circuit-PSI and OT. Note that although PSI-InnerProd has some additional procedures such as MultShare, they can be realized by OT as explained in Section II-C, and hence we only need to assume the security of OT.

V. Extension to Private Set Union

This section provides how to instantiate a private set union (PSU) protocol via circuit-PSI. Private Set Union (PSU) enables two parties, each holding inputs X and Y, respectively, to privately compute the union set $X \cup Y$ without revealing their individual inputs. The ideal functionality $\mathcal{F}_{\mathsf{PSU}}$ is described in Fig. 17. We particularly emphasize that the

Parameters: A set size n and a data field $\mathbb{F}$.
Input: $\mathcal{S}$ with a set X and an associated data $D^X = \{d_x^X \in \mathbb{F} : x \in X\}$, and $\mathcal{R}$ with a set Y and an associated data $D^Y = \{d_y^Y \in \mathbb{F} : y \in Y\}$
Protocol:

1) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\mathsf{cPSI}}$ with associated data. After the execution, $\mathcal{S}$ and $\mathcal{R}$ receive $\mathbf{b}_s \in \{0,1\}^\beta, \mathbf{a}_s^Y \in \mathbb{F}^\beta$ and $\mathbf{b}_r \in \{0,1\}^\beta, \mathbf{a}_r^Y \in \mathbb{F}^\beta$, respectively.
2) For each $i \in [\beta]$, $\mathcal{S}$ and $\mathcal{R}$ invoke B2A-OT twice:
   - In the first B2A-OT execution, $\mathcal{S}$ inputs $b_{s,i} \in \{0,1\}$ and $a_{s,i}^Y \in \mathbb{F}$, and $\mathcal{R}$ inputs $b_{r,i} \in \{0,1\}$ to have arithmetic shares of $b_i \cdot a_{s,i}^Y$.
   - The second B2A-OT execution is performed with reversed role, where $\mathcal{S}$ inputs $b_{s,i} \in \{0,1\}$, and $\mathcal{R}$ inputs $b_{r,i}\{0,1\}$ and $a_{r,i}^Y \in \mathbb{F}$ to have arithmetic shares of $b_i \cdot a_{r,i}^Y$.
   - By locally adding the two shares, the two parties obtain arithmetic shares of $b_i \cdot a_i^Y$.
3) $\mathcal{S}$ defines $\mathbf{d}^X \in \mathbb{F}^\beta$ such that for each $i \in [\beta]$
$$d_i^X = \begin{cases} d_x^X & \text{if } \mathsf{idx}(x) = i, \\ 0 & \text{otherwise.} \end{cases}$$
4) For each $i \in [\beta]$,
   a) $\mathcal{S}$ computes $a_{s,i} = a_{s,i}^Y \cdot d_i^X$.
   b) $\mathcal{S}$ and $\mathcal{R}$ invoke MultShare on $\mathcal{S}$'s input d_i^X and $\mathcal{R}$'s input $a_{r,i}^Y$, to obtain arithmetic shares of $a_{r,i}^Y \cdot d_i^X$.
   c) $\mathcal{S}$ adds its share of $a_{r,i}^Y \cdot d_i^X$ to $a_{s,i}$.
5) $\mathcal{S}$ computes $a_s = \sum_i a_{s,i}$ and $\mathcal{R}$ computes $a_r = \sum_i a_{r,i}$.
6) $\mathcal{S}$ sends a_s to $\mathcal{R}$.
7) $\mathcal{R}$ outputs $a_s + a_r$.

Fig. 16: Our PSI-InnerProd protocol based on circuit-PSI

receiver $\mathcal{R}$ with an input set Y is not allowed to learn any partial membership information, say $\mathbf{1}(y \in X)$.

Parameters: A set size n
Input: $\mathcal{S}$ with a set X and $\mathcal{R}$ with a set Y
Functionality: Output $X \cup Y$ to $\mathcal{R}$.

Fig. 17: Ideal functionality $\mathcal{F}_{\mathsf{PSU}}$ of PSU

A. Our Private Set Union Protocol

In this subsection, we provide key ideas for designing a PSU protocol based on circuit-PSI protocols and then present the full description of our PSU construction.

1) Sending $X - Y$: The starting idea for building PSU from circuit-PSI is as follows. After invoking the circuit-PSI protocol, the two parties have Boolean shares $\mathbf{b}_s$ and $\mathbf{b}_r$ such that $\mathbf{b} = \mathbf{b}_s \oplus \mathbf{b}_r$. Then $\mathcal{S}$ reorders its set X using the index map idx and performs OT with $\mathcal{R}$ to send $X - Y$ in the following: $\mathcal{S}$ takes two OT messages $m_{b_{s,i}} = x_i, m_{1-b_{s,i}} = \perp$ as inputs, while $\mathcal{R}$ takes $b_{r,i}$ as the OT choice. After the execution of the OT protocol, $\mathcal{R}$ obtains x_i if $b_i = 1$, (i.e., $x_i \in Y$). Otherwise, $\mathcal{R}$ obtains $\perp$. Thus, by executing OT for all $i \in [\beta]$, $\mathcal{R}$ finally obtains $X - Y$. The above procedure is illustrated in Fig. 18.

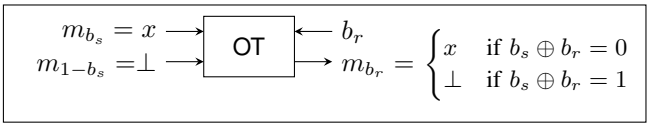

Fig. 18: OT to send $X - Y$ from membership Boolean shares

2) Shuffling Boolean Shares: However, a direct application of the above OT-based $X - Y$ sending results in information leakage, due to the nature of cuckoo hashing nature in circuit-PSI. For example, if $\mathcal{R}$ finally obtains an element x in Y_i during the i-th OT execution, it implies that the membership indicator b_i was 0. This actually means that $\mathcal{R}$'s i-th bin Y_i does not intersect with X, which violates the security requirement of PSU, as $\mathcal{R}$ learns $\mathbf{1}(y \in X) = 0$ for $y \in Y_i$.

To prevent this leakage, we shuffle the membership vector $\mathbf{b}$ so that $\mathcal{R}$ cannot associate its hash bin Y_i with the final OT result. At the same time, $\mathcal{S}$ must know the shuffling because it needs to follow the correct position of each element $x \in X$ for the final OT. To achieve this, we utilize the permute-and-share (P&S) functionality defined in Fig. 19.

Parameters: A vector length n and an item length ℓ
Input: $\mathcal{S}$ with a permutation $\pi : [n] \to [n]$ and $\mathcal{R}$ with input vector $\mathbf{t} \in (\{0,1\}^\ell)^n$
Functionality: Sample a random vector $\mathbf{s} \xleftarrow{\$} (\{0,1\}^\ell)^n$. Then send $\mathbf{s}$ to $\mathcal{S}$ and $\pi(\mathbf{t}) \oplus \mathbf{s}$ to $\mathcal{R}$.

Fig. 19: Ideal functionality $\mathcal{F}_{\mathsf{P\&S}}$ of permute-and-share

Given Boolean share vectors $\mathbf{b}_s$ and $\mathbf{b}_r$, the two parties first execute P&S with $\mathcal{S}$'s input permutation π and $\mathcal{R}$'s input $\mathbf{b}_r$. This process results in secret shares of $\pi(\mathbf{b}_r)$, denoted as $(\mathbf{s}, \pi(\mathbf{b}_r) \oplus \mathbf{s})$. $\mathcal{S}$ who holds $\mathbf{b}_s$ and π, can locally compute $\pi(\mathbf{b}_s) \oplus \mathbf{s}$, which becomes its share of $\pi(\mathbf{b})$ since

$$(\pi(\mathbf{b}_s) \oplus \mathbf{s}) \oplus (\pi(\mathbf{b}_r) \oplus \mathbf{s}) = \pi(\mathbf{b}_s \oplus \mathbf{b}_r) = \pi(\mathbf{b}).$$

The above procedure for shuffling Boolean shares is illustrated in Fig. 20.

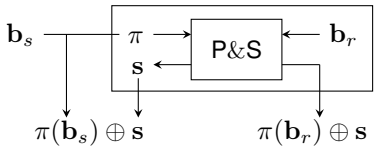

Fig. 20: Shuffling Boolean shares using P&S

Finally, since $\mathcal{S}$ know the permutation π, it can locally reorder the OT messages so that each element of $X - Y$ is correctly transmitted using the protocol described in Fig. 18. A pictorial explanation of the procedure for shuffling Boolean shares and then applying the OT is provided in Fig. 21.

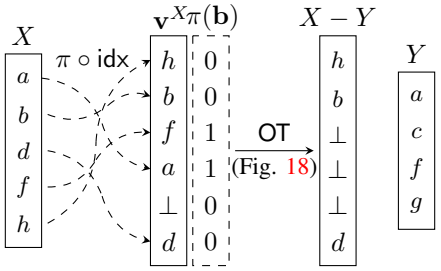

Fig. 21: Shuffling Boolean shares and applying the final OT in Fig. 18

The full description of our PSU protocol is presented in Fig. 22.

Remark 2. *A concurrent work [18], [32], which also proposed circuit-PSI based PSU constructions, reported an additional leakage issue in cuckoo-hashing-based PSU protocols. We adopt the countermeasure presented in [18], which preprocesses the PSU inputs with OPRF. For a detailed explanation, please refer to [18].*

Parameters: A set size n
Input: $\mathcal{S}$ with a set X and $\mathcal{R}$ with a set Y
Protocol:

1) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\mathsf{OPRF}}$:
   - Both parties agree on an OPRF output length ℓ.
   - $\mathcal{S}$ gets a PRF key k and $\mathcal{R}$ gets $\bar{Y} = \{F_k(y) : y \in Y\}$.
2) $\mathcal{S}$ locally computes $\bar{X} = \{F_k(x) : x \in X\}$.
3) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\mathsf{cPSI}}$ and obtain secret shares $\mathbf{b}_s$ and $\mathbf{b}_r$, respectively, such that $\mathbf{b} = \mathbf{b}_r \oplus \mathbf{b}_s \in \{0,1\}^\beta$. $\mathcal{S}$ additionally obtains a mapping $\mathsf{idx} : X \to [\beta]$.
4) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\mathsf{P\&S}}$:
   - $\mathcal{S}$ takes a random permutation π on $[\beta]$ as an input and $\mathcal{R}$ takes as an input $\mathbf{b}_r \in \{0,1\}^\beta$
   - $\mathcal{S}$ receives $\mathbf{s} \in \{0,1\}^\beta$, while $\mathcal{R}$ receives $\mathbf{c}_r = \pi(\mathbf{b}_r) \oplus \mathbf{s} \in \{0,1\}^\beta$.
5) $\mathcal{S}$ locally computes $\mathbf{c}_s = \pi(\mathbf{b}_s) \oplus \mathbf{s}$ and $\mathbf{v}^X \in \mathbb{F}^\beta$ such that
$$v_i^X = \begin{cases} x & \text{if } \pi \circ \mathsf{idx}(x) = i \\ \bot & \text{otherwise.} \end{cases}$$
6) $\mathcal{R}$ initializes $Z = \emptyset$. Then, for each $i \in [\beta]$,
   a) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\mathsf{OT}}$:
      - $\mathcal{S}$ takes two OT messages $m_{c_{s,i}} = v_i^X$ and $m_{1-c_{s,i}} = \bot$ as inputs and $\mathcal{R}$ takes $c_{r,i}$ as the OT choice.
      - After the execution of $\mathcal{F}_{\mathsf{OT}}$, $\mathcal{R}$ obtains the OT result z_i.
   b) $\mathcal{R}$ adds z_i to Z if $z_i \neq \bot$.
7) $\mathcal{R}$ outputs $Y \cup Z$.

Fig. 22: Our PSU Protocol

B. Correctness and Security

For the correctness, we need to check that the final OT works well. More precisely, for $x \in X$, let $\pi \circ \mathsf{idx}(x) = i$. Then we need to check that $c_{s,i}$ and $c_{r,i}$ that used for the i-th OT correctly indicates the membership of $x \in X$: Since it holds that $c_{s,i} \oplus c_{r,i} = b_{\pi^{-1}(i)}$ and

$$b_{\pi^{-1}} = \begin{cases} \mathbf{1}(x \in Y) & \text{if } \mathsf{idx}(x) = \pi^{-1}(i), \\ 0 & \text{otherwise,} \end{cases}$$

we can confirm the correctness.

The security argument is similar to previous protocols: every information is secret-shared during the protocol. In particular, the membership information $\mathbf{b}$ remains secret-shared during the protocol so that both parties cannot know the exact membership result (corresponding each bin). Although $\mathcal{R}$ finally knows the permuted information $\pi(\mathbf{b})$, it provides no information beyond $|X \cap Y|$ to $\mathcal{R}$, because π is only known to $\mathcal{S}$. Therefore, assuming the sub-protocols that regards secret-shared computation (precisely, circuit-PSI and OT and P&S protocol) is securely realized, our proposed PSU reveals no additional information than the desired output: $X - Y$.

VI. Performance Evaluations

This section evaluates the performance of the PSO protocols presented in this paper through experimental results.

A. Submodule Instantiations

Although the main bodies of the PSO protocols are described in a fully modular fashion, their performance depends on the concrete instantiation of each submodule. We introduce recent constructions of each submodule that our implementation is also based on.

1) Oblivious Transfer: Recall that the key idea of extending circuit-PSI into PSO is to perform OT with properly defined messages. Thus, our protocols typically require a large number of OT intstances. We perform OT in the preprocessed model [33], which divides OT into the setup and online phases: First, in the setup phase, two parties generate *random OT* instances. Then, in the online phase, the sender passes the encryption of the desired message using the random OT instances as ephemeral secrets and thus the receiver can decrypt one of the message ciphertexts using its random OT output. This enables us to perform OT with message bit-length of ℓ using a total communication cost of $2\ell + 1$.

Furthermore, we rely on the *OT extension* protocol [34], which efficiently generates a large number of random OT instances. Particularly, we utilize the state-of-the-art OT extension protocol from [35], which generates a bulk of random OT instances with an almost negligible communication cost.

2) Circuit-PSI: The primary component of our protocol is clearly circuit-PSI, which was realized using the state-of-the-art construction from [12]. As a notable point, the construction involves a procedure that generates a Boolean share of the equality result ($\mathbf{a} = \mathbf{b}$), where each party holds $\mathbf{a}$ and $\mathbf{b}$, respectively. This procedure is typically executed using a generic two-party computation protocol, such as a garbled

circuit or the GMW protocol. However, it is worthwhile to consider this in more detail. The aforementioned preprocessed OT model with (random) OT extension can also be applied in this setting, allowing the (time-consuming) random OT generation to be shifted into the setup phase. In fact, as we will see later in Section VI-C, this two-party computation part occupies a significant portion of the computational burden in circuit-PSI. This observation means that such a heavy computational burden can be done in the setup phase, which could be benefical for some practical use cases.

3) Permute & Share: Our PSU protocol further requires the P&S functionality in Fig. 19. We consider the protocol in [36], which has a complexity of $O(\ell n \log n)$ and is based on the Benes network representation of given permutation, executing each switch in the network using OT. Again, the OTs can be executed in the preprocessed model by shifting the random OT generation to the setup phase.

Moreover, there is a recent report on an improved P&S protocol [37] with linear complexity $O(n\ell)$. However, since our PSU protocol considers only the special case of $\ell = 1$, the concrete performance of the protocol in [37] is actually worse than that of [36] for our cases. Therefore, we choose not to include it in our implementation.

4) Oblivious PRF: We also require the OPRF functionality described in Fig. 2 for our PSU protocol, where we adapt the state-of-the-art construction from [12]. Recall that the OPRF was introduced to prevent the information leakage identified by [18]. However, this leakage occurs only when the receiver obtains the set difference $X - Y$, and hence is not applicable to other PSO functionalities. As a result, this OPRF preprocessing is necessary only for the PSU protocol.

B. Implementation Setup

We implement our protocols in C++.[2] For CPSI and OPRF, we used the public implementation [38] of the construction in [12]. For OT extension, we used the implementation from [39], and the P&S realization of the protocol in [36] is adapted from [7].

All experiments were tested on Intel Xeon Silver 4208 running at 2.10GHz CPU and 128GB RAM, with the sender and receiver each running on a single thread. The network environment between two parties are simulated by Linux tc command, where the LAN network was set to a 10Gbps bandwidth and a 0.1ms RTT latency, and the WAN network was set to a 50Mbps bandwidth with an 80ms RTT latency.

- As basic parameters, we set the computational security parameter to $\kappa = 128$ and the statistical security parameter to $\lambda = 40$. These values determine the internal parameters of OPRF, circuit-PSI and P&S, as well as their concrete costs. We refer to [11] for the details.
- We set the bit-length of each input set item to 128 and the bit-length of associated value to 32. Note that the circuit-PSI cost is almost independent of the input item length, while these lengths affect the final OT phase, which has a cost of $O(\ell_D \cdot n)$.

[2]The source code of our implementation is available at https://github.com/yonghaason/volepsi.

C. Micro-Benchmark from Breakdown

Table I shows the running time and communication cost of our PSU protocol for input sets of size $n = 2^{20}$.

TABLE I: Performance breakdowns of our PSU protocol with $n = 2^{20}$ where each submodule is instantiated as in Section VI-A.

		Comm. (MB)	LAN (s)	WAN (s)
Setup	Circuit-PSI	1.46	58.4	58.8
	P&S	0.29	11.1	11.3
	OT	0.17	0.42	0.43
	Total	1.92	69.4	70.0
Online	OPRF	22.75	0.75	1.83
	Circuit-PSI	95.29	4.21	16.6
	P&S	10.40	3.46	3.56
	OT	41.60	1.46	6.54
	Total	170.4	8.97	28.5

One can see that the majority of the circuit-PSI running time is spent generating large amounts of correlated randomness (e.g. random OTs). It should be remarked that the DH-based protocols cannot fully take advantage of this property.

D. Comparison with other PSO protocols

Table II presents the experimental results for several PSO protocols including ours.

a) Comparison with RPMT approaches: First, assuming that the setup phase (which is independent of the input) can be separated, one can focus solely on the online running time–that is, the time required after the actual input is provided. In this case, compared to the previous best results in [8], PSI-Card and PSI-Sum achieve an improvement of approximately 9.92–13×, PSU improves by about 4.8–5.2×, and PSI-InnerProd shows a 47–56× improvement relative to [17] over the LAN network. Even when taking into account the total running time, which includes both the setup and online phases, our results are comparable to those of [8], with PSI-InnerProd still achieving more than a 5-fold improvement in the running time.

Meanwhile, our circuit-PSI based protocols generally incur a slightly higher communication cost, which leads to somewhat degraded performance in the WAN environment. As a result, when considering the total running time–that is, both the setup and online phases–there are cases where existing RPMT-based approaches outperform ours.

In fact, there is another factor that makes the performance of our protocols worse in slower network: Our protocol is based on a little bit more complicated construction than that in [8] which is based on a relatively simple DH-based approach. Hence, our protocols require more interaction between the two parties. As a result, under high-latency conditions, the increased number of interactions further slows down the overall protocol execution. This effect is especially pronounced for small set sizes. In this case, not only do the time for local computation and the actual transmission information matter, but the latency of each interaction also has a considerable impact. As a result, there remains a significant difference in online times between the protocol in [8] and our approach when $n = 2^{20}$, but when $n = 2^{16}$, the impact of latency

TABLE II: Communication costs and running times of several PSO protocols

		$n = 2^{16}$					$n = 2^{20}$				
		Comm. (MB)	LAN (s)		WAN (s)		Comm. (MB)	LAN (s)		WAN (s)	
			Setup	Online	Setup	Online		Setup	Online	Setup	Online
PSI-Card	[7]	55.5	-	6.56	-	24.6	1030	-	84.9	-	284.6
	[8]	4.45	-	3.77	-	4.91	71.3	-	60.2	-	72.7
	Ours	7.05	3.15	0.38	3.65	3.51	99.4	57.8	4.64	59.6	16.8
PSI-Sum	[22]†	43.2	-	176.3	-	186.3	(Not reported)				
	[8]†	5.75	-	3.98	-	5.65	95.3	-	63.2	-	81.1
	Ours	7.45	3.15	0.39	3.51	3.49	105.9	58.1	4.61	59.6	16.8
PSU	[8]	6.48	-	3.87	-	5.43	103.3	-	60.2	-	78.4
	[16]	137.4	0.88	3.73	4.48	16.6	2430	5.89	43.5	25.0	200.3
	Ours	12.7	3.58	0.66	4.09	5.21	172.3	68.3	8.96	70.6	28.5
PSI-InnerProd	[17]†*	17.2	-	25.9	-	-	275.2	-	414.7	-	-
	Ours	20.2	3.94	0.55	4.72	6.7	307.3	73.6	7.4	76.1	62.3
PSI-Threshold	Ours	7.26	3.13	0.38	3.50	5.56	99.56	58.1	4.67	58.5	25.2

The numbers for our main comparison target [8] are obtained by running their implementation [40] on our machine.

†: Protocols in [7], [8], [17] additionally reveal the intersection cardinality besides the target result (Sum or InnerProd), whereas ours can only reveal the target result.

*: The implementation of [17] is not publicized, so we just take the numbers from the original paper, which is only known for a LAN environment.

becomes even more pronounced, leading to nearly identical online times.

As for both security and functionality perspectives, the key distinction between our approach and that of [8] lies in the unintended leakage of cardinality information, specifically $|X \cap Y|$. As discussed above, RPMT-based PSO protocols inevitably reveal the exact cardinality $|X \cap Y|$, whereas our approach avoids this leakage. This difference arises because, in RPMT, the receiver directly obtains a membership vector, whereas in our protocol, the receiver learns only Boolean shares of the membership information. At first glance, the distinction between directly receiving the membership vector and learning only its Boolean shares may seem minor.

However, this subtle difference has significant implications for PSOs. For instance, prior RPMT-based constructions [8], [15] cannot be applied to building PSI-Threshold, where the receiver should learn only a single bit indicating whether $|X \cap Y| > t$ for some threshold t. The reason is that, after completing the RPMT step, the receiver already possesses full membership information, making it impossible to restrict the learned information to a simple threshold comparison. A similar issue arises in PSI-Sum, where the receiver should obtain only $\sum_{x_i} d^X_{x_i}$–the sum of the associated values for elements in the intersection–yet prior approaches leak additional unintended information.

Another example can be observed in the realizations of PSO functionalities, such as PSU and PSI-Sum, as shown in [8, Fig. 12]. In these constructions, each receiver inevitably learns $|X \cap Y|$. However, according to their intuitive definitions–that no party should learn any information beyond the intended output–this realization slightly deviates from their original security expectations. For a more rigorous interpretation, such unintended information leakage should be regarded as a security issue. As a result, to the best of our knowledge, this work presents the first concrete implementation of PSI-Threshold, PSI-Sum, and PSU protocols using modern PSO techniques.

b) Private Set Unions: We provide additional remarks on PSU protocols. Recall that an RPMT-based protocol [8] demonstrates quite a decent performance, but Jia et al. [16] identified a security vulnerability in RPMT-based PSU protocols [6]–[8], [15]. At the same time, they proposed a new PSU protocol [16] to eliminate unintended security leakage, but this comes at the cost of significantly increased communication overhead.

From a security perspective, our PSU protocol achieves the same level of security as [16], since each party learns only secret shares of the output derived from the circuit-PSI protocol throughout execution. In terms of performance, our protocol reduces the communication cost by approximately $14\times$ compared to [16], while maintaining a similar level of computational cost. Therefore, our protocol achieves performance comparable to recent PSU protocols in [8], [18], demonstrating both practicality and efficiency.

Compared to the concurrent and independent work of [18], which also explores a modular construction for PSU, our approach theoretically eliminates the during-execution leakage identified in [16]. Our construction ensures that the P&S functionality returns permuted Boolean shares of the input, whereas the combine-and-permute functionality in [18] produces a permuted bit vector rather than secret shares. This subtle difference implies that, in our protocol, no party ever learns the intermediate membership results directly, whereas in [18], unintended information leakage could still occur due to the structure of their permutation mechanism.

VII. Conclusion

This paper presents several PSO protocols based on a black-box circuit-PSI framework, including PSI-Sum, PSI-Cardinality, PSI-Threshold, PSI-InnerProd, and PSU. Our

constructions eliminate unintended security leakages while preserving efficiency. In particular, by separating execution into offline and online phases, our approach allows for further optimizations in specific settings. By enhancing both security and computational performance, our work provides a strong foundation for future advancements in PSOs.

REFERENCES

[1] M. Wu and T. H. Yuen, "Efficient unbalanced private set intersection cardinality and user-friendly privacy-preserving contact tracing," in *USENIX Security 2023*, J. A. Calandrino and C. Troncoso, Eds. USENIX Association, 2023, pp. 283–300.

[2] D. Morales, I. Agudo, and J. López, "Private set intersection: A systematic literature review," *Comput. Sci. Rev.*, vol. 49, p. 100567, 2023.

[3] Y. Jia, S. Sun, H. Zhou, J. Du, and D. Gu, "Shuffle-based private set union: Faster and more secure," in *USENIX Security 2022*, K. R. B. Butler and K. Thomas, Eds. USENIX Association, 2022, pp. 2947–2964.

[4] L. Kissner and D. X. Song, "Privacy-preserving set operations," in *CRYPTO 2005*, ser. Lecture Notes in Computer Science, V. Shoup, Ed., vol. 3621. Springer, 2005, pp. 241–257.

[5] A. Davidson and C. Cid, "An efficient toolkit for computing private set operations," in *ACISP 2017 Part II*, ser. Lecture Notes in Computer Science, J. Pieprzyk and S. Suriadi, Eds., vol. 10343. Springer, 2017, pp. 261–278.

[6] V. Kolesnikov, M. Rosulek, N. Trieu, and X. Wang, "Scalable private set union from symmetric-key techniques," in *ASIACRYPT 2019 Part II*, ser. Lecture Notes in Computer Science, S. D. Galbraith and S. Moriai, Eds., vol. 11922. Springer, 2019, pp. 636–666.

[7] G. Garimella, P. Mohassel, M. Rosulek, S. Sadeghian, and J. Singh, "Private set operations from oblivious switching," in *PKC 2021 Part II*, ser. Lecture Notes in Computer Science, J. A. Garay, Ed., vol. 12711. Springer, 2021, pp. 591–617.

[8] Y. Chen, M. Zhang, C. Zhang, M. Dong, and W. Liu, "Private set operations from multi-query reverse private membership test," in *PKC 2024 Part III*, ser. Lecture Notes in Computer Science, Q. Tang and V. Teague, Eds., vol. 14603. Springer, 2024, pp. 387–416.

[9] V. Kolesnikov, R. Kumaresan, M. Rosulek, and N. Trieu, "Efficient batched oblivious PRF with applications to private set intersection," in *ACM CCS 2016*, E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi, Eds. ACM, 2016, pp. 818–829.

[10] B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, "PSI from PaXoS: Fast, malicious private set intersection," in *EUROCRYPT 2020 Part II*, ser. Lecture Notes in Computer Science, A. Canteaut and Y. Ishai, Eds., vol. 12106. Springer, 2020, pp. 739–767.

[11] P. Rindal and P. Schoppmann, "VOLE-PSI: fast OPRF and circuit-PSI from vector-OLE," in *EUROCRYPT 2021 Part II*, ser. Lecture Notes in Computer Science, A. Canteaut and F. Standaert, Eds., vol. 12697. Springer, 2021, pp. 901–930.

[12] S. Raghuraman and P. Rindal, "Blazing fast PSI from improved OKVS and subfield VOLE," in *ACM CCS 2022*, H. Yin, A. Stavrou, C. Cremers, and E. Shi, Eds. ACM, 2022, pp. 2505–2517.

[13] A. Bienstock, S. Patel, J. Y. Seo, and K. Yeo, "Near-optimal oblivious key-value stores for efficient PSI, PSU and volume-hiding multi-maps," in *USENIX Security 2023*, J. A. Calandrino and C. Troncoso, Eds. USENIX Association, 2023, pp. 301–318.

[14] Y. Huang, D. Evans, and J. Katz, "Private set intersection: Are garbled circuits better than custom protocols?" in *NDSS 2012*. The Internet Society, 2012.

[15] C. Zhang, Y. Chen, W. Liu, M. Zhang, and D. Lin, "Linear private set union from multi-query reverse private membership test," in *USENIX Security 2023*, J. A. Calandrino and C. Troncoso, Eds. USENIX Association, 2023, pp. 337–354.

[16] Y. Jia, S. Sun, H. Zhou, and D. Gu, "Scalable private set union, with stronger security," in *USENIX Security 2024*, D. Balzarotti and W. Xu, Eds. USENIX Association, 2024.

[17] K. Chida, K. Hamada, A. Ichikawa, M. Kii, and J. Tomida, "Communication-efficient inner product private join and compute with cardinality," in *ASIA CCS 2023*, J. K. Liu, Y. Xiang, S. Nepal, and G. Tsudik, Eds. ACM, 2023, pp. 678–688.

[18] G. R. Chandran, T. Schneider, M. Stillger, and C. Weinert, "Concretely efficient private set union via circuit-based PSI," *Cryptology ePrint Archive 2024/1494*, 2024, to appear at ASIACCS 2025.

[19] C. Meadows, "A more efficient cryptographic matchmaking protocol for use in the absence of a continuously available third party," in *IEEE S&P 1986*. IEEE Computer Society, 1986, pp. 134–137.

[20] B. Pinkas, T. Schneider, and M. Zohner, "Faster private set intersection based on OT extension," in *uUSENIX Security 2014*, K. Fu and J. Jung, Eds. USENIX Association, 2014, pp. 797–812.

[21] N. Chandran, D. Gupta, and A. Shah, "Circuit-PSI with linear complexity via relaxed batch OPPRF," *Proc. Priv. Enhancing Technol.*, vol. 2022, no. 1, pp. 353–372, 2022.

[22] M. Ion, B. Kreuter, A. E. Nergiz, S. Patel, S. Saxena, K. Seth, M. Raykova, D. Shanahan, and M. Yung, "On deploying secure computing: Private intersection-sum-with-cardinality," in *IEEE EuroS&P 2020*. IEEE, 2020, pp. 370–389.

[23] T. Lepoint, S. Patel, M. Raykova, K. Seth, and N. Trieu, "Private join and compute from PIR with default," in *ASIACRYPT 2021 Part II*, ser. Lecture Notes in Computer Science, M. Tibouchi and H. Wang, Eds., vol. 13091. Springer, 2021, pp. 605–634.

[24] K. B. Frikken, "Privacy-preserving set union," in *ACNS 2007*, ser. Lecture Notes in Computer Science, J. Katz and M. Yung, Eds., vol. 4521. Springer, 2007, pp. 237–252.

[25] C. Hazay and K. Nissim, "Efficient set operations in the presence of malicious adversaries," in *PKC 2010*, ser. Lecture Notes in Computer Science, P. Q. Nguyen and D. Pointcheval, Eds., vol. 6056. Springer, 2010, pp. 312–331.

[26] M. Blanton and E. Aguiar, "Private and oblivious set and multiset operations," in *ASIACCS 2012*, H. Y. Youm and Y. Won, Eds. ACM, 2012, pp. 40–41.

[27] O. Goldreich, S. Micali, and A. Wigderson, "How to play any mental game or a completeness theorem for protocols with honest majority," in *ACM STOC 1987*, A. V. Aho, Ed. ACM, 1987, pp. 218–229.

[28] B. Pinkas, T. Schneider, O. Tkachenko, and A. Yanai, "Efficient circuit-based PSI with linear communication," in *EUROCRYPT 2019 Part III*, ser. Lecture Notes in Computer Science, Y. Ishai and V. Rijmen, Eds., vol. 11478. Springer, 2019, pp. 122–153.

[29] Y. Arbitman, M. Naor, and G. Segev, "Backyard cuckoo hashing: Constant worst-case operations with a succinct representation," in *IEEE FOCS 2010*. IEEE Computer Society, 2010, pp. 787–796.

[30] B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, "SpOT-Light: Lightweight private set intersection from sparse OT extension," in *CRYPTO 2019 Part III*, ser. Lecture Notes in Computer Science, A. Boldyreva and D. Micciancio, Eds., vol. 11694. Springer, 2019, pp. 401–431.

[31] K. Han, S. Kim, and Y. Son, "Private computation on common fuzzy records," *Proc. Priv. Enhancing Technol.*, vol. 2025, no. 1, pp. 567–583, 2025.

[32] K. Liu, X. Li, and T. Takagi, "Review the cuckoo hash-based unbalanced private set union: Leakage, fix, and optimization," in *ESORICS 2024 Part II*, ser. Lecture Notes in Computer Science, J. García-Alfaro, R. Kozik, M. Choras, and S. K. Katsikas, Eds., vol. 14983. Springer, 2024, pp. 331–352.

[33] G. Asharov, Y. Lindell, T. Schneider, and M. Zohner, "More efficient oblivious transfer and extensions for faster secure computation," in *ACM CCS 2013*, A. Sadeghi, V. D. Gligor, and M. Yung, Eds. ACM, 2013, pp. 535–548.

[34] Y. Ishai, J. Kilian, K. Nissim, and E. Petrank, "Extending oblivious transfers efficiently," in *CRYPTO 2003*, ser. Lecture Notes in Computer Science, D. Boneh, Ed., vol. 2729. Springer, 2003, pp. 145–161.

[35] S. Raghuraman, P. Rindal, and T. Tanguy, "Expand-convolute codes for pseudorandom correlation generators from LPN," in *CRYPTO 2023 Part IV*, ser. Lecture Notes in Computer Science, H. Handschuh and A. Lysyanskaya, Eds., vol. 14084. Springer, 2023, pp. 602–632.

[36] P. Mohassel and S. S. Sadeghian, "How to hide circuits in MPC an efficient framework for private function evaluation," in *EUROCRYPT 2013*, ser. Lecture Notes in Computer Science, T. Johansson and P. Q. Nguyen, Eds., vol. 7881. Springer, 2013, pp. 557–574.

[37] S. Peceny, S. Raghuraman, P. Rindal, and H. Shah, "Efficient permutation correlations and batched random access for two-party computation," *IACR Cryptol. ePrint Arch. 2024/547*, 2024.

[38] Visa-Research, "volepsi: Efficient private set intersection base on vole," 2022. [Online]. Available: https://github.com/Visa-Research/volepsi

[39] P. Rindal, "libOTe: A fast, portable, and easy to use oblivious transfer library," 2022. [Online]. Available: https://github.com/osu-crypto/libOTe

[40] Y. Chen, "Kunlun: A modern crypto library," 2023. [Online]. Available: https://github.com/yuchen1024/Kunlun